Capstone Project – Full VAPT Cycle

1. Introduction

This capstone project demonstrates a complete Vulnerability Assessment and Penetration Testing (VAPT) cycle conducted in a controlled lab environment. The objective is to identify security vulnerabilities in the target system, exploit them to demonstrate risk impact, and collect evidence for documentation.
The assessment was conducted using industry-standard penetration testing tools including Nmap, Metasploit Framework, Burp Suite, SQLMap, and Wireshark.
The testing was performed in a local virtual lab environment to ensure safe and ethical testing practices.

---

2. Lab Environment

The penetration testing lab environment consists of an attacker machine and a vulnerable target machine.

Component	Description
Attacker Machine	Kali Linux
Target Machine	Metasploitable 2
Target IP Address	192.168.206.139
Testing Network	Virtual Lab Environment

The attacker system (Kali Linux) was used to perform reconnaissance, vulnerability scanning, exploitation, and evidence collection against the vulnerable Metasploitable2 machine.

---

3. Penetration Testing Methodology

The testing process followed standard penetration testing phases:

1. Reconnaissance
2. Vulnerability Scanning
3. Exploitation
4. Web Application Testing
5. Post-Exploitation
6. Evidence Collection and Reporting

Each phase is documented in the following sections.

---

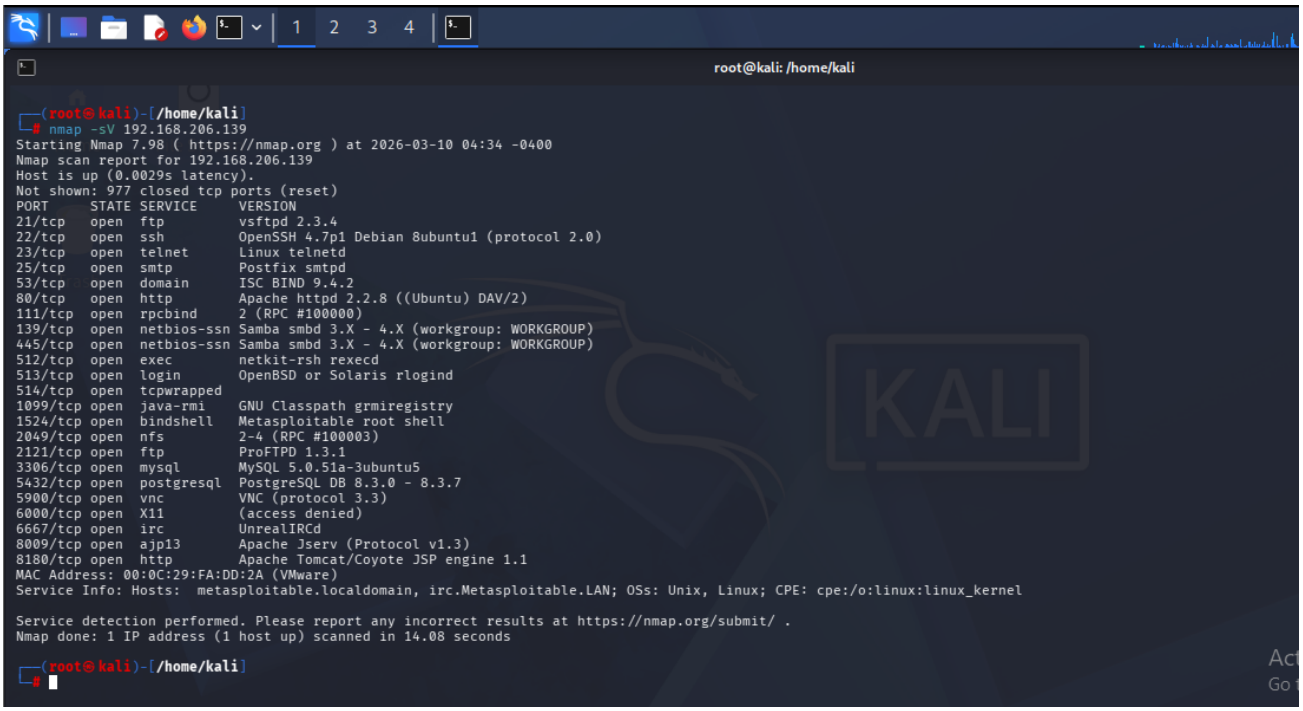
4. Reconnaissance

Reconnaissance is the process of gathering information about the target system to identify possible attack vectors.
Tool Used
Nmap – Network Mapper
Command Used
nmap -sV 192.168.206.139
This command scans the target machine and identifies running services and their versions.
Result

The scan identified multiple open ports and services running on the target machine.

Port	Service
21	FTP
22	SSH
80	HTTP
3306	MySQL

These services were further analyzed to identify known vulnerabilities.



Description: Screenshot showing the Nmap scan results with open ports and detected services.

5. Vulnerability Identification

Based on the services discovered during reconnaissance, several vulnerabilities were identified.

Service	Vulnerability
VSFTPD 2.3.4	Backdoor Command Execution
Web Application	SQL Injection
Web Application	Cross-Site Scripting (XSS)

These vulnerabilities were confirmed using penetration testing tools and manual testing techniques.

6. Exploitation

The exploitation phase involves using specialized tools to gain unauthorized access to the vulnerable system.

Tool Used

Metasploit Framework

Exploit Module

exploit/unix/ftp/vsftpd_234_backdoor

Steps Performed

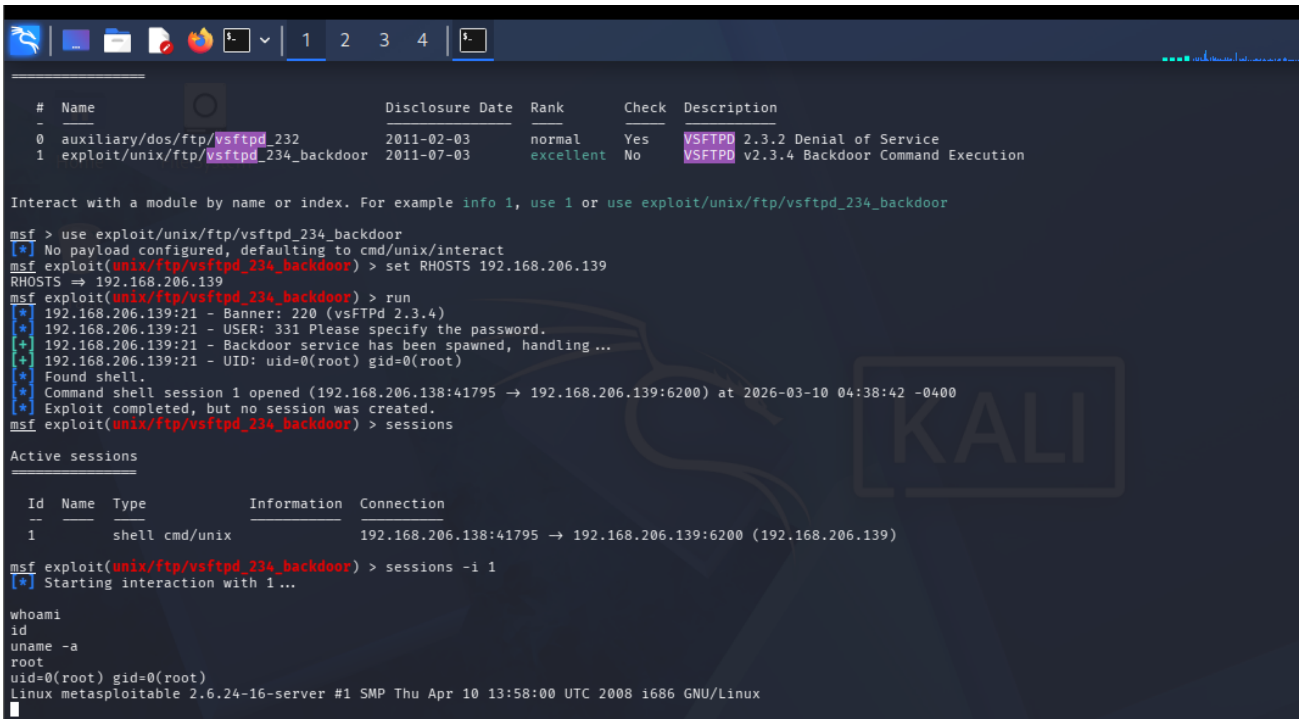
1. Launch Metasploit Framework
2. Search for the VSFTPD exploit module
3. Configure the target IP address
4. Execute the exploit

Commands Used

msfconsole
use exploit/unix/ftp/vsftpd_234_backdoor
set RHOSTS 192.168.206.139
run

Result

The exploit successfully opened a command shell on the target machine.
uid=0(root) gid=0(root)
This indicates that root-level access was obtained.



Description: Screenshot showing successful exploitation and root access obtained via Metasploit.

7. Web Application Testing

The target system hosts several intentionally vulnerable web applications such as DVWA (Damn Vulnerable Web Application).
The web application was tested for common vulnerabilities.

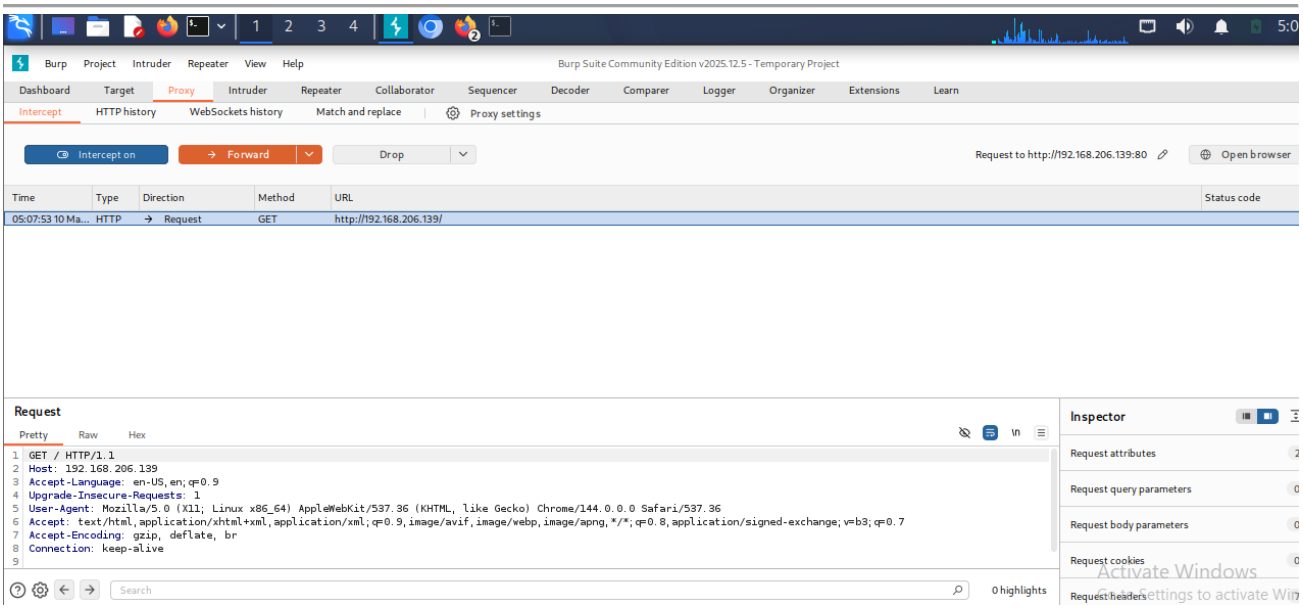
Tools Used

Burp Suite
SQLMap

7.1 Intercepting Web Requests

Burp Suite was used as a proxy to intercept HTTP requests between the browser and the web application.

This helps analyze request parameters and identify injection points.



Description: Screenshot showing Burp Suite intercepting HTTP requests from the web application.

7.2 SQL Injection Testing

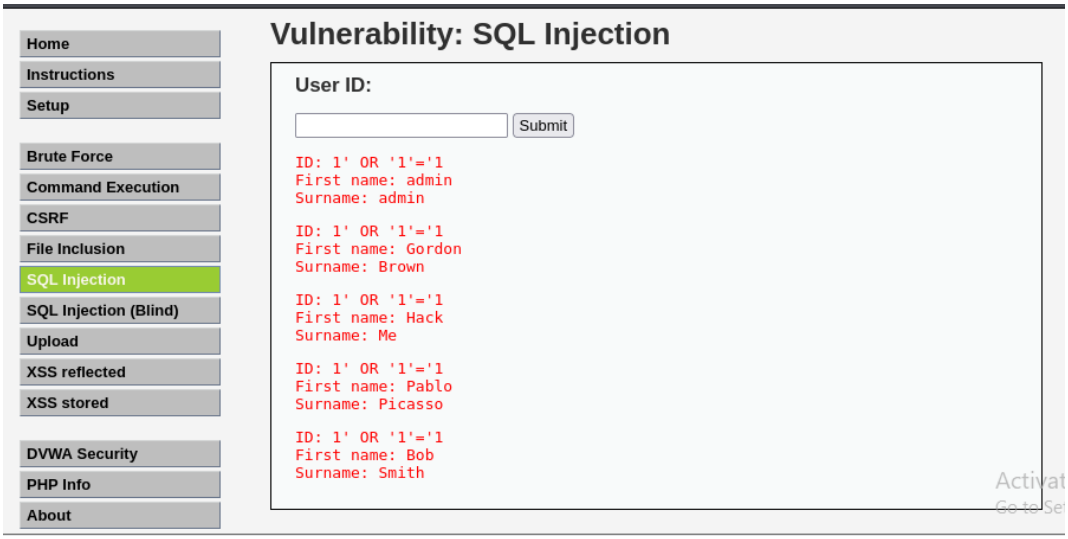
SQL Injection vulnerabilities allow attackers to manipulate database queries.

SQLMap was used to automate the detection and exploitation of SQL Injection vulnerabilities.

Example Command

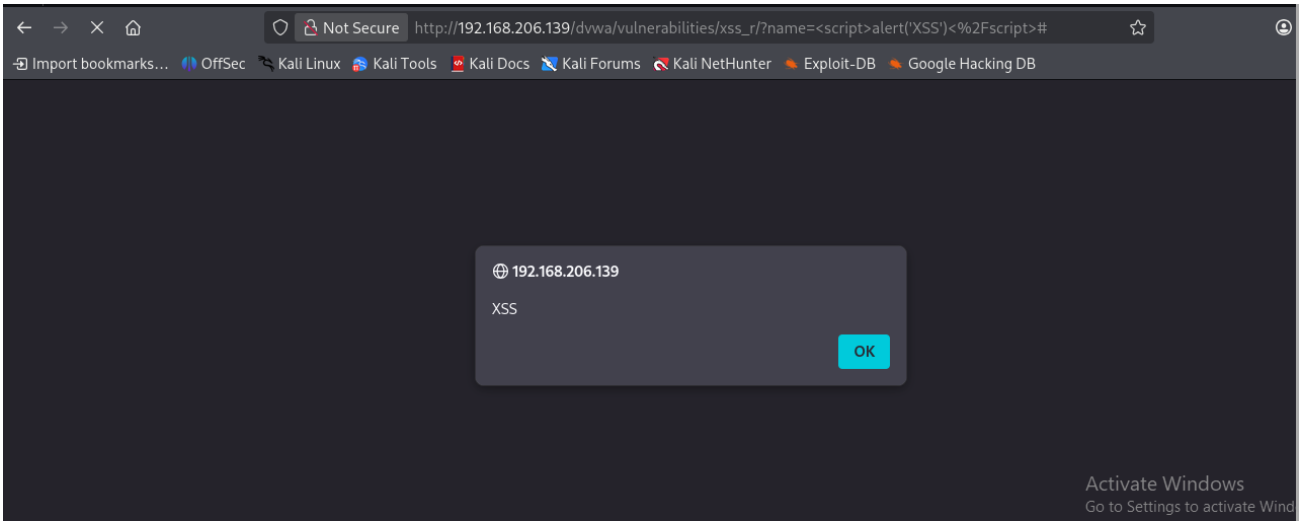
sqlmap -u "http://192.168.206.139/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" --dbs

This command attempts to extract database information from the vulnerable application.



Description: Screenshot demonstrating SQL Injection testing on the DVWA application.

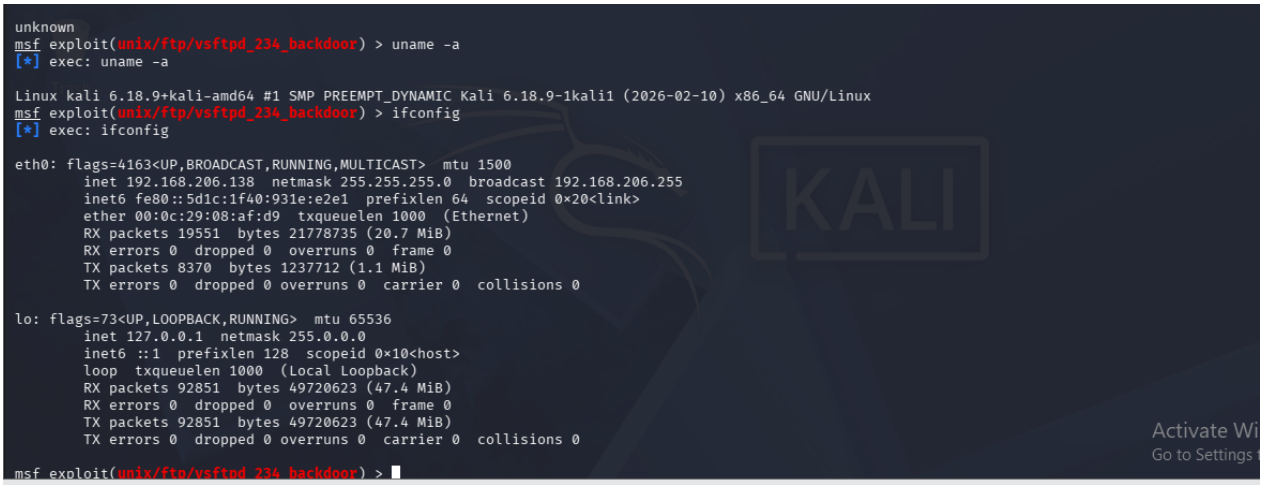
7.3 Cross-Site Scripting (XSS)
Cross-Site Scripting occurs when user input is not properly sanitized.
Example payload used during testing:
<script>alert('XSS')</script>
When executed successfully, the browser displays a JavaScript alert.



Description: Screenshot showing successful execution of the XSS payload.

8. Post-Exploitation

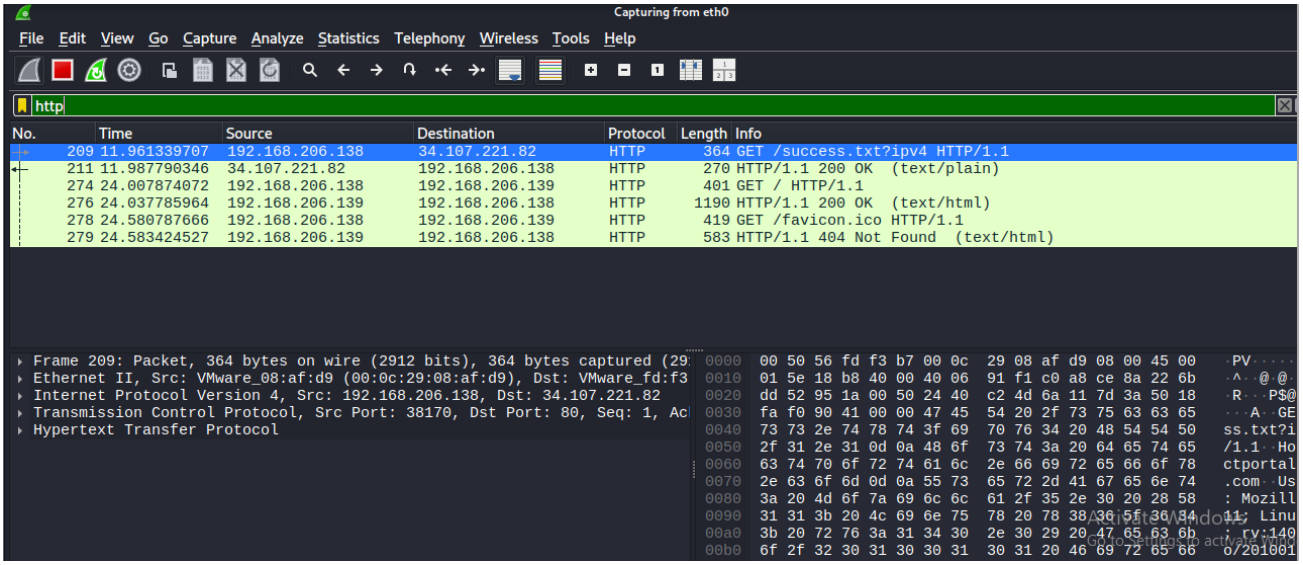
After gaining access to the system, additional information was collected from the target.
Commands Used
whoami
id
uname -a
ifconfig
These commands were used to verify user privileges and gather system information.



Description: Screenshot showing system information collected from the compromised system.

9. Network Traffic Capture

Network traffic between the attacker and target system was captured using Wireshark. Wireshark allows analysis of HTTP and TCP packets to observe communication patterns during the attack.



Description: Screenshot showing captured HTTP packets in Wireshark.

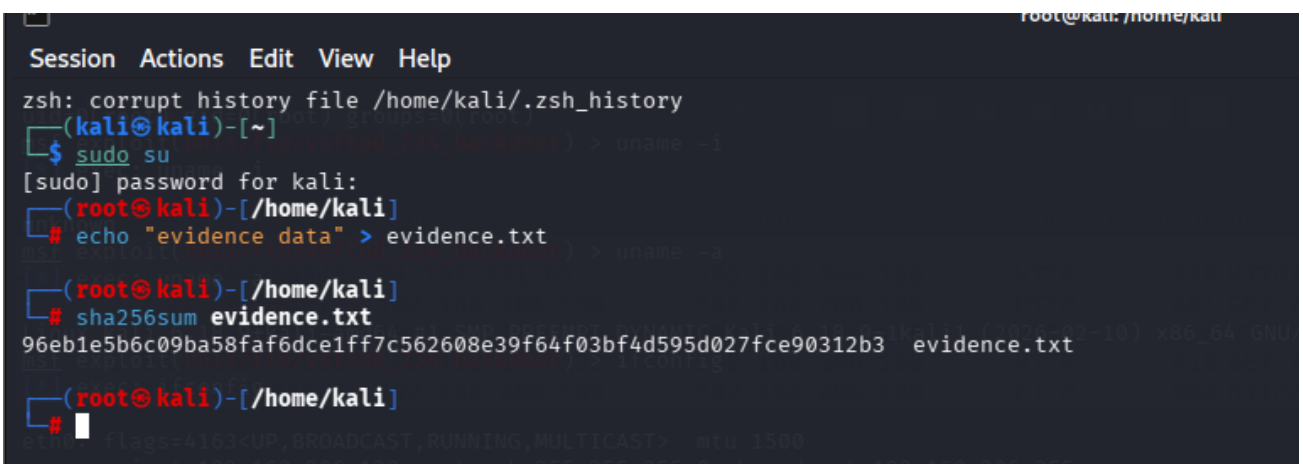
10. Evidence Integrity Verification

To ensure that collected evidence remains unchanged, file hashing was performed using the SHA256 algorithm.

Command Used

sha256sum evidence.txt

The generated hash value ensures the integrity of the evidence file.



Description: Screenshot showing SHA256 hash generation for evidence verification.

11. Key Findings

Vulnerability	Severity	Description
VSFTPD Backdoor	Critical	Allows remote root access
SQL Injection	High	Allows attackers to access database information
Cross-Site Scripting	Medium	Allows execution of malicious scripts in the browser

12. Recommendations

To improve the security posture of the target system, the following recommendations are suggested:

1. Update or patch vulnerable services such as VSFTPD.
2. Implement secure input validation to prevent SQL Injection.
3. Apply output encoding techniques to mitigate Cross-Site Scripting.
4. Disable unnecessary services and ports.
5. Conduct regular vulnerability assessments and security testing.

13. Conclusion

The penetration testing exercise successfully identified multiple critical vulnerabilities within the target environment. Exploitation of the VSFTPD backdoor allowed attackers to gain root-level access to the system. Additionally, web application vulnerabilities such as SQL Injection and Cross-Site Scripting were identified.

Proper patch management, secure coding practices, and regular security assessments are essential to prevent such vulnerabilities from being exploited in real-world environments.